

A Time Window based Reinforcement Learning Reward for Test Case Prioritization in Continuous Integration

Zhaolin Wu

Beijing University of Chemical Technology
Chaoyang District, Beijing, China
wuzhaolin96@outlook.com

Zheng Li*

Beijing University of Chemical Technology
Chaoyang District, Beijing, China
lizheng@mail.buct.edu.cn

Yang Yang

Beijing University of Chemical Technology
Chaoyang District, Beijing, China
yyyilian8695@163.com

Ruilian Zhao

Beijing University of Chemical Technology
Chaoyang District, Beijing, China
rlzhao@mail.buct.edu.cn

ABSTRACT

Continuous integration refers to the practice of merging the working copies of all developers into the mainline frequently. Regression testing for each merge is characterized by continually changing test suite, limited execution time, and fast feedback, which demands new test optimization techniques. Reinforcement learning is introduced for test case prioritization to save computing resources in continuous integration environment, where a reasonable reward function is highly important for learning strategy, since the process of reinforcement learning is a reward-guided behavior. **In this paper, APHFW, a novel reward function is proposed by using partial historical information of test cases effectively for fast feedback and cost reduction.** The experiments are based on three open-source data sets, and the results show that the proposed reward function is more cost-effect than other reinforcement learning rewards in continuous integration environment.

CCS CONCEPTS

• **Computer systems organization** → **Embedded systems**; *Redundancy*; Robotics; • **Networks** → Network reliability.

KEYWORDS

continuous integration, reinforcement learning, reward function, test case prioritization

ACM Reference Format:

Zhaolin Wu, Yang Yang, Zheng Li, and Ruilian Zhao. 2018. A Time Window based Reinforcement Learning Reward for Test Case Prioritization in Continuous Integration. In *Proceedings of Internetware 2019: 11th Asia-Pacific Symposium on Internetware (Internetware 2019)*. ACM, New York, NY, USA, 6 pages.

*Corresponding author

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Internetware 2019, October 28–29, 2019, Fukuoka, Japan

© 2018 Association for Computing Machinery.

ACM ISBN 978-1-4503-9999-9/18/06...\$15.00

DOI: 10.1145/3361242.3361258

1 INTRODUCTION

In software engineering, Continuous Integration (CI) is an agile, automated and periodic integration test process that automatically merges developers' work to a mainline. Continuous integration and continuous delivery can help to reduce repetitive processes and save time and cost, where workers can get test feedback from regression testing faster. Companies such as Google, Apple have adopted CI automatic tools in their software development processes, such as Jenkins¹, which is an open source CI tool written in Java. GitLab CI² is part of GitLab, which is also widely used in software development.

Continuous integration and continuous delivery require high quality automated testing, which is a prerequisite to check whether the software works as expected. A failed test usually means there is a new error in the last version and the software version will be determined to be inoperable. Automatic testing is of importance in continuous integration context. Various techniques have been proposed for regression testing optimization, such as Test Suite Reduction (TSR) [16, 19], Regression Test Selection (RTS) [18, 20, 23] and Test Case Prioritization (TCP) [3, 6–8, 10].

Test selection aims at identifying the subset of test suite that have to execute in some criteria, such as fault detection ability, execution cost. Test prioritization reorders test cases to satisfy a certain target, such as revealing failures as soon as possible. In our experimental evaluation, test case prioritization is used in resource-constrained environments to approximate real world situation, i.e., we prioritize test cases firstly and select part of the set by a certain total available execution time.

Different from traditional regression testing, there are several challenges in continuous integration context, and some prior techniques are not applicable [12]. First, as the software version continues to iterate, the scale of test suites will grow faster. Memon et al. [14] observed linear growth in both code committing rate and test suite scales, resulting in huge expenses to execute all test suites. Second, the building of system in CI environments could be spectacularly frequent. Engineers have to fix several times until the arrival of the correct version. For example, coverage-based TCP requires the data about code coverage, which collected in the pre-processing period [2, 15]. In continuous integration environments,

¹<https://jenkins.io/>

²<https://about.gitlab.com/product/continuous-integration/>

the preprocessing periods may not be available because intervals between two CI cycles are too short and the frequency is so high.

Machine learning based techniques are proposed to deal with the challenges, where Reinforcement Learning (RL) is used for online automatic test case prioritization. For the first time, Spieker et al. [21] applied the RL framework to test case prioritization and selection in CI environments and implemented a lightweight automatic learning TCP algorithm. The process of reinforcement learning is a reward-guided behavior in which an agent interacts with the environment and learns in a ‘trial and error’ manner with the goal of maximizing the reward of the agent [1]. A positive or negative reward value is calculated based on historical results by a reward function, and used for the next action decided by the agent.

The determination of an appropriate reward value can influence the outcome of reinforcement learning strongly. If the reward function is not well designed, it is difficult for the agent to learn the desired goal. In the Spieker’s method [21], only the test case execution result of recent one generation is used to calculate the reward value in the reward function. It is not comprehensive enough to consider the latest generation of cycle, which losing potential valuable history information. He proposed a reward method [9] with all history information of test cases and showed a higher error detection ability than Spieker’s method. However, it is unnecessary consider all history because some early information may become obsolete. In regression testing, there is a hypothesis that test cases previously failed are likely to fail in the followed tests, i.e., the recently executed test cases are more important than the test cases that were executed earlier. Thus a time window based approach is adopted in the reward function to extract more valuable historical information from recent test case to earlier ones.

In this paper, a novel reinforcement learning reward function, Average Percentage of Historical Failure with time Window (APHFW), is proposed for test case prioritization in CI environments. APHFW utilizes a time window to filter recent historical information to calculate reward value. We experimentally analyzed the impact of different window sizes reward functions on the fault detecting level and time cost, and a suitable window size is identified to satisfy relatively optimal error detection capability as well as time cost. We have evaluated our approach on three industrial CI data sets. The results show that different sizes of time window reward functions have different time costs and fault detection abilities when they applied to those data sets. Furthermore, we found that by selecting five as the size of the time window, the APHFW reward function performs optimally because it both maintains good fault detection ability and effectively saves time costs.

In summary, the contributions of the paper are as follows:

- A novel time-window based reward function is presented for reinforcement learning, in which effective historical information of test cases is extracted for test case prioritization in CI environments.
- Experiments based on three industrial data sets illustrated that the proposed reward function can reduce time cost significantly compared with full history information based reward for test case prioritization in CI environments, while the performance is not regressive.

- An appropriate range of window size is further experimental determined with the statistical analysis of the historical distribution of test case failures of three industrial data sets.

The remainder of the paper organized as follows: Section 2 provides related work about testing in CI and background about using RL in CI. Section 3 describes the APHFW reward function in detail. Section 4 presents the design and results of the study and discusses the results. Section 5 concludes our work and describes our future work.

2 BACKGROUND AND RELATED WORK

The objective of the test case prioritization technique is to reveal regression faults earlier by reordering test cases with certain criteria. In each CI cycle, a test suite can be sorted based on the history results of test cases, which is the coverage information of testing codes, or combined with the time cost of test cases to reduce time overhead [5]. In the past few decades, many approaches have been proposed to ameliorate TCP techniques. However, continuous integration is used widely to integrate the committing codes from hundreds of developers periodically and to build and test their works automatically, which brings new challenges. Traditional regression testing optimization techniques cannot be used in CI environments directly and simply, because the code churn is too frequent and the versions of test suite keep iterating.

Elbaum Rothermel et al.[5] proposed an approach based on test suite failure and execution history to prioritize on test suites-level in 2014. Liang et al. [11] used a similar algorithm on commits-level which is different from others. Experiments showed that the proposed algorithms can find errors faster and were suitable for the development of large industrial programs, such as Google data sets. However, these algorithms were only compared with the manual approach without others’ methods. Marijan et al. [13] investigated the effect of history length on the ordering of test cases. The results showed that the time window of different sizes can significantly change the performance of regression tests. They compared their method with a random algorithm and manual algorithm which is still traditional regression test optimization rather than machine learning based techniques. The criteria of test case prioritization are also investigated, where Yoo et al. [22] reviewed the test cases before submission with the goal of maximizing dependency coverage, defect detection, and minimizing test time. This method was also validated on Google data sets and reduced test time by 33%-82%.

Busjaeger et al. [4] used machine learning to train a priority-based test selection model. This model was implemented by some existing heuristic algorithms, such as test coverage, textual similarity, test failure history and test age. Their empirical evaluation is based on a large real-world data set, Salesforce³, and the results showed that the ranking model had a good performance on fault detection. Spieker et al. [21] proposed a test case prioritization and selection approach, RETECS (Enhanced Test Case Selection), based on reinforcement learning (RL) in a CI environment. This method has attracted our attention that it can dynamically learn and automatically sort the test cases, so we did a detailed study of the RETECS. To compare the advantages and disadvantages of our method, we use the same RL framework.

³<https://www.salesforce.com/>

3 TIME WINDOW BASED REINFORCEMENT LEARNING REWARD

In this section, we will introduce the reward function which based on effective history information in details, named Average Percentage of Historical Failure with time Window (APHFW).

In each CI cycle, a set of test cases which prepare to be executed called a test schedule. It aims at finding faults earlier and reducing test workload without compromising quality. The reward function reflects these goals and evaluates the performance of a test schedule to reinforce the agent that we may obtain an effect test cases queue in the following cycles. The reliability of the evaluation of the reward function determines whether our approach can assess test cases accurately.

There are several ways to adopt partial historical information strategies. The first one is to randomly sample some historical information. This method is very simple, but it is hard to filter the important part. We hope to save the valuable historical information to obtain better reward value but this method is different with our intention. The second method is to use historical information with recent generations. We believe that the recent history is more important than the early test for sorting techniques for test cases. Moreover, in CI environments, test cases are not always the same. That means with the version update, some test cases will be added into the test suite for requirement while some others may be obsolete because of the varying version. Therefore, we use a time window, moves as the cycle changes, to obtain partial historical information. For further analysis, a window with a fixed step size and interval can be selected, the step size indicates the number of information acquisitions per time, and the interval indicates the distance between the two execution results. For example, the interval is 1 means we will adopt continuous historical information without spacing. A varying size of window brings different performance for TCP, we will choose a window with a dynamically variable step size or interval in the future work. From the perspective of simplicity and minimization of effective information, we adopt a window sliding method with a fixed step size and interval of size 1. In next section, we analyze the appropriate window steps by experimental statistics.

A reasonable idea to utilize preceding execution results more effect is that we can filter out outdated and redundant history results. So we propose a time-effective reward function, named APHFW (Average Percentage of Historical Failure with time Window), to optimize the reinforcement learning framework. It filters out unwelcome and invalid execution history and regards recent failure history as a good reward. The definition of APHFW reward function is as follows.

$$APHFW_i(t) = 1 - \frac{\sum_{j=1}^m R_j}{win(n, W) \times m} + \frac{1}{2 \times win(n, W)} \quad (1)$$

Let i be the i_{th} CI cycle. TS be the test suite, and t be a test case ($teTS$). Let $win(n, W)$ be the W size of the time window of the history results, while n means a test case has n execution results in the i_{th} CI cycle ($W \leq n \leq i$). The value of W will be determined through experiment and evaluation in the next section. Let m be the number of failed execution results from $win(n, W)$. R_j denotes the countdown order of the last j_{th} failure of t . From the reward function we know that the value of APHFW ranges from 0 to 1.

The higher the APHFW value is, the greater the probability that t failed in the recent historical execution, and the more likely t will be detected a failure in the next CI cycle.

4 EXPERIMENTAL EVALUATION

In this section, we present an experimental evaluation of the APHFW reward function. Section 4.1 describes an overview of the data sets adopted in implemented context. In Section 4.2 the independent variables and the dependent variables are presented. In Section 4.3 we show and evaluate the experiment results.

First we need to determine an appropriate range of the $win(n, W)$, different sizes of time window may cause different results. We need to point out that, in special situation, when W equals to 1 or n , the reward function is TF or APHF, respectively. TF is claimed by Spieker [21] and APHF is proposed by He [9]. Then within the evaluation of the APHFW reward function, we investigate whether the size of $win(n, W)$ influences the effectiveness and the time cost of test case prioritization in CI or not. We will also compare our approach to other authors' reward methods.

RQ1: How to choose an appropriate window size range to make our experiment more reasonable?

RQ2: How the fault detection ability and the time cost of APHFW change with the different sizes of historical information window? How does it compare to other reinforcement learning techniques in the same industrial data sets?

RQ3: Can we identify an optimal size of time window W to satisfy high error detection ratio and low time cost simultaneously?

4.1 Subjects

Table 1: Overview of Industrial Datasets

Data set	Test cases	CI cycles	Results	Failure rate
Paint Control	114	312	25,594	19.36%
IOF/ROL	2,086	320	30,319	28.43%
GSDTSR	5,555	336	1,260,617	0.25%

For the implementation of the experimental evaluation, we use data sets from ABB Robotics Norway ⁴, Paint Control and IOF/ROL, and Google Shared Dataset of Test Suite Results(GSDTSR) ⁵. They are both real-world industrial continuous integration data sets.

ABB is a leading supplier of industrial robots, and robot software, equipment, and complete application solutions, the data sets of Paint Control and IOD/ROL are used to test industrial robots and software development. Each data set contains historical information on testing results and over 300 CI cycles. For those two data sets, each CI cycle contains test suites in one day.

GSDTSR is an open resource of Google shared data set provided by Elbaum, which consists of over 3.5 million execution results of test suite among 30 days. After filtering and selecting, Spieker et al. adopt over 1.2 million verdicts and split them into hourly intervals, GSDTSR also has over 300 CI cycles. For comparison, we adopted it at the same situation.

⁴<http://new.abb.com/products/robotics>

⁵<https://code.google.com/archive/p/google-shared-dataset-of-test-suite-results>

Table 1 describes the overview of the data sets, which contains the number of test cases, the CI cycles amount, the number of verdicts and the failure rate of test case executions. The test suite consists of different test cases in each cycle. Each CI cycle represents an integration test that it comprises the verdicts of the test cases. We use those verdicts to calculate the reward value for sorting test cases in next CI cycle. The failure rate indicates the proportion of failures in the total number of test case executions.

4.2 Variables and Measures

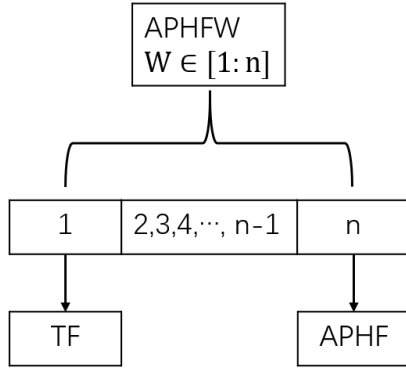


Figure 1: The links between APHFW and other reward functions

4.2.1 Independent Variables. The primary independent variable is the size of time window W of the APHFW reward function. After preliminary processing (details in RQ1), we set 14 windows, the set of $W = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 15, 20, 25, n]$. Figure 1 classifies the relationship among three reward functions (APHFW, APHF and TF). When APHFW just calculate the latest ($W=1$) or the whole history information ($W=n$), APHFW is equivalent to TF ($W=1$) or APHF ($W=n$), respectively. Actually, APHFW is a general form, which contains TF and APHF reward, TF and APHF are special forms of APHFW.

4.2.2 Dependent Variables. The first dependent variable represents the effectiveness of fault detection. That is the Average Percentage of Faults Detected (APFD), which as evaluation metric in this work. The traditional APFD only considers the case where all test suites are executed. However, to simulate the CI cycle's time limit, a fixed percentage of 50% of the required time is used [21]. APFD could not stand for the performance, so we replace APFD with the Normalized APFD (NAPFD)[17], which is an extension of the former. It can reflect the proportion of errors detected by the test suite to all errors and is suitable for the existence of test case selection. Of course, if all errors are detected, the NAPFD is the same as APFD ($p=1$). The formula is as follows:

$$NAPFD(TS_i) = p - \frac{\sum_{t \in TS_i^{fail}} rank(t)}{|TS_i^{fail}| \times |TS_i|} + \frac{p}{2 \times |TS_i|} \quad (2)$$

With $p = \frac{|TS_i^{fail}|}{|TS_i^{total, fail}|}$ represents the position of the t_{th} failed

test case in the test sequence TS , let $|TS_i^{fail}|$ be the total number of test cases that failed in the test suite sequence, $|TS_i|$ indicates the total number of test cases in the test sequence, and $|TS_i^{total, fail}|$ denotes the total number of test cases that failed in the test suite. Since continuous integration is a dynamic process, NAPFD is tested once per integration cycle. We calculate the average value of the list of NAPFD to describe it.

The second dependent variable is the time cost, which represents the cost-effectiveness of reward functions. It consists of calculating reward values through the historical results, selecting and resorting test cases by the agent.

4.3 Experimental Results and Analysis

4.3.1 RQ1: How to choose an appropriate window size range to make our experiment more reasonable? As described above, the method of sliding window is adopted to obtain the recent historical information. However, the range of window size that we set to implement the experiment still needs to explore by preliminary test. We collected the historical error distribution of several industrial data.

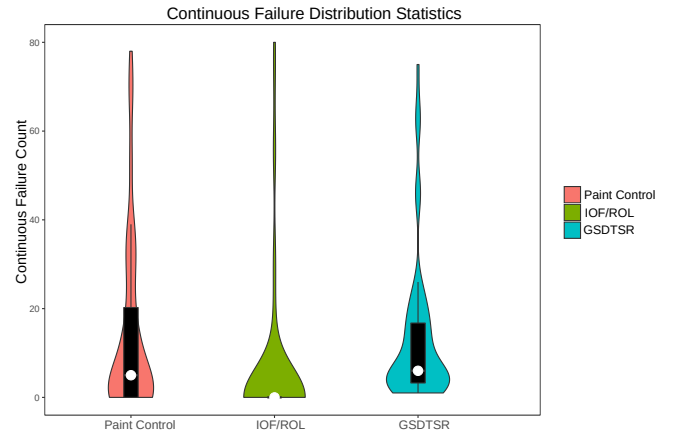


Figure 2: the distribution of the failed test cases execution history in three data sets

Figure 2, a picture of violin plots, shows the distribution of the failed test cases execution history in three data sets. The violin plot is used to display the data distribution and its probability density. Through those violin plots, it's clear that their lower bodies are wider than the upper bodies. We found that most of the continuous error counts are less than 10, and the median is around 5. In other words, this picture shows the density distribution of consecutive error intervals for the respective test cases of the three data sets, and the density distribution is concentrated within 10. So we assume that the step size of the sliding window which is less than 10 is reasonable. To further verify this hypothesis, we set up 10 sets of small-step experiments when the window size is less than or equal to 10. The window size starts from 1 to 10, which is $[1, 2, 3, 4, 5,$

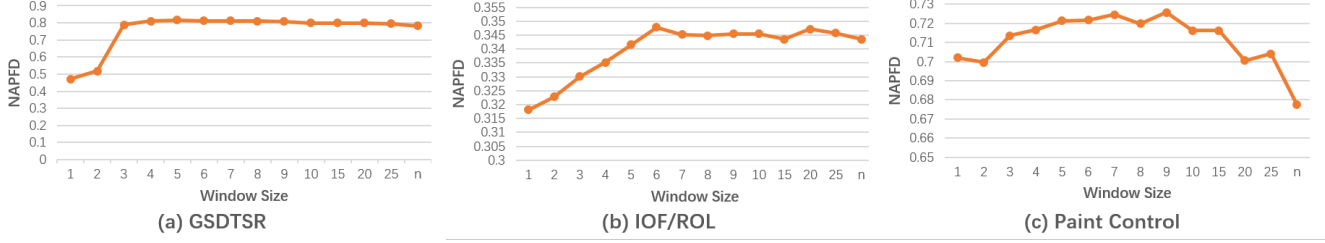


Figure 3: The NAPFD of the reward functions with different window sizes

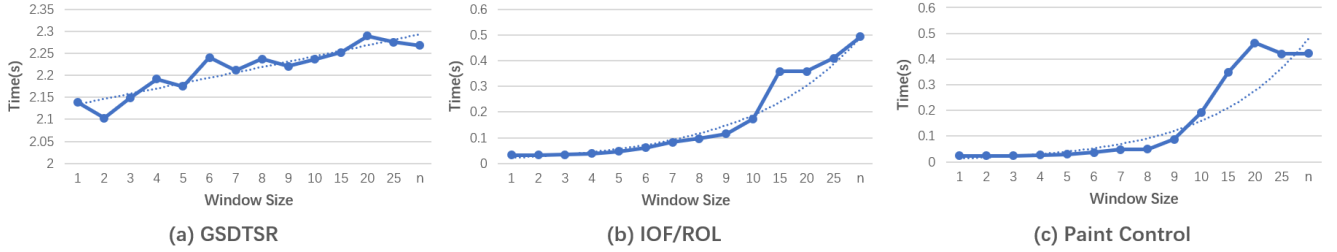


Figure 4: The time cost of calculating reward values with different window sizes

6, 7, 8, 9, 10]. When the window size is greater than 10, several sets of large-step experiments are set and the span is 5, which is [15, 20, 25, n], where n indicates that the window is using the whole history information. As a result, we adopted a list of window sizes $win(n, W) = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 15, 20, 25, n]$ to avoid considering useless sizes.

4.3.2 RQ2: The description of outcomes of APHFW. We investigate the performances of APHFW with different W. Figures 3 describes the distribution of the average NAPFD values for the different window sizes about three data sets. The x-axis represents time windows of various sizes; the y-axis represents the NAPFD values. Figure 4 shows the distribution of the average duration values. The x-axis represents different time window sizes W, and the y-axis denotes the duration values. The dotted line indicates the exponential trend. We emphasize that TF (W=1) and APHF (W=n) are the particular forms of APHFW (W from 1 to n).

In three data sets, when W is small, which contains TF (W=1), the fault detection ability is poor with negligible time cost. When W is larger, APHFW has better fault detection performance with higher time cost. For GSDTSR, its NAPFD keeps flat when W is more than 3, while the time cost has an upward trend. Similar to IOF/ROL, when W is equal to 1 to 6, NAPFD gets growth and it maintains stability. However, in Paint Control, it has a wavy motion. When W equals n, which means it considers all execution history, the performance is worse than other reward functions (TF and APHFW).

For three data sets in Figure 4, it can be seen that the time cost increases with the increase of W in most cases. Indicated by the dotted line, APHFW has an exponential increase of time cost with increasing of W. When the window size W is between 1 and 9, the

time overhead cost is not significant. However, when W is between 9 to 15, the time cost increases obviously in IOF/ROL and Paint Control. The growth rate of time cost under GSDTSR data is 4.3%, the growth rate under IOF/ROL data is 965.8%, and the growth rate under Paint Control data is 1326.8%. These values calculate through the time costs of TF and APHF (when W=1 and W=n, respectively).

In general, those APHFW reward functions which utilize recent historical information have better results than TF and have not weaker performances than APHF. It means that APHFW considers enough but not redundant historical information to arrange a reasonable prioritization list, which can detect fault more quickly. In the following subsection, we make discussion and identify an appropriate W to resolve RQ3.

4.3.3 The Discussion of the experiment. Such as TF, it takes only one generation history information into account, which loses many valuable experiences. It explains that when the window size is small, the error detection capability performs weakly. However, when APHFW takes all history information, which is APHF, its performance becomes worse in Paint Control and IOF/ROL, while GSDTSR's APHF has same NAPFD with APHFW. We have analyzed the fault distribution of those data sets, we find that the failure rate of GSDTSR is only 0.25%. Compared with other data sets, only a few test cases failed in GSDTSR. From RQ1 we know that most of the continuous error counts are less than 10 but GSDTSR has more than 100 test results per cycle. It explains why APHFW has the same performance when W is more than 3. The failure rate of ABB's data is high, while the number of test cases is small. It is the main reason why the time cost increased so high. If the APHFW window is infinitely large (the historical information is so big), as long as the test case t has a failure once, then each time t is executed,

the reward function needs to be calculated once, so it takes much time. In the Paint Control data set between 200 and 250 cycles, a large number of failed test cases were repeatedly added to the test suite manually. Those test cases were fixed but if we consider all history information to calculate reward function, it will predict the wrong conclusion. That's why APHF has a weak performance in Paint Control.

To address RQ3, we hope to find an optimal W^* that it can be not only practical but also efficient. IOF/ROL data and Paint Control data have a short time overhead when W is less than or equal to 5, while their fault detection abilities are efficient. In GSDTSR the performance is useful when the time window size is between 5 and 10, while the time overhead cost has no apparent fluctuation. We determine that the APHFW function with a window size of $W^*=5$ can guarantee better error detection ability and save more time cost. In future work, we need to consider more different databases to summary the rule of time window size.

5 CONCLUSION AND FUTURE WORK

In this paper, we focused on the influence of different sizes of historical information on test case prioritization using reinforcement learning in CI environments, and two aspects are evaluated, the time cost and the fault detection ability. We investigated how can historical information improve the effectiveness of test case prioritization effectively. A novel reward function is proposed by utilizing partial execution history of test cases for RL technique to create a better prioritization scheme. The reward function named the Average Percentage of Historical failure with time Window (APHFW). In order to get a reasonable window size W for optimally information filtering, we analyzed the distribution of the failed test cases history. By taking a small step when W is small and a big step when W is large, we sat up a serious of groups to do the experiment on three industrial data sets.

The work shows that (1) Compared with reward function with one generation historical information, the fault detection ability of APHFW reward function is better. (2) Compared with reward function with whole execution history, the time cost of APHFW reward function is lower without compromising the quality. (3) When W is 5 the APHFW function performs the best and it can be used as a reference object in industry.

The future work will focus on: (1) Analyzing more industrial data sets to find a general window size W . (2) Using dynamic sliding window to automatically catch useful historical information and ignore the uselessness. (3) Optimizing the neural network agent to obtain better performance.

ACKNOWLEDGMENTS

The work described in this paper is supported by the National Natural Science Foundation of China under Grant No.61872026, 61672085 and 61702029, and the Fundamental Research Funds for the Central Universities(XK1802-4).

REFERENCES

[1] Andrew G Barto and Richard S Sutton. 1998. *Reinforcement Learning: An Introduction*. The MIT Press.

[2] Árpád Beszédes, Tamás Gergely, Lajos Schrettnér, Judit Jász, László Langó, and Tibor Gyimóthy. 2012. Code coverage-based regression test selection and prioritization in WebKit. In *2012 28th IEEE international conference on software maintenance (ICSM)*. IEEE, 46–55.

[3] Yi Bian, Zheng Li, Ruilian Zhao, and Dunwei Gong. 2017. Epistasis Based ACO for Regression Test Case Prioritization. *IEEE Transactions on Emerging Topics in Computational Intelligence* 1, 3 (2017), 213–223.

[4] Benjamin Busjaeger and Tao Xie. 2016. Learning for test prioritization: an industrial case study. In *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering*. ACM, New York, NY, USA, 975–980. <https://doi.org/10.1145/2950290.2983954>

[5] Sebastian Elbaum, Gregg Rothermel, and John Penix. 2014. Techniques for improving regression testing in continuous integration development environments. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*. ACM, 235–245.

[6] S. Haidry and T. Miller. 2013. Using Dependency Structures for Prioritization of Functional Test Suites. *IEEE Transactions on Software Engineering* 39 (02 2013), 258–275. <https://doi.org/10.1109/TSE.2012.26>

[7] Dan Hao, Lingming Zhang, Lu Zhang, Gregg Rothermel, and Hong Mei. 2014. A Unified Test Case Prioritization Approach. *ACM Trans. Softw. Eng. Methodol.* 24, 2, Article 10 (Dec. 2014), 31 pages. <https://doi.org/10.1145/2685614>

[8] M. J. Harrold, R. H. Untch, G. Rothermel, and C. Chu. 2001. Prioritizing Test Cases For Regression Testing. *IEEE Transactions on Software Engineering* 27 (10 2001), 929–948. <https://doi.org/10.1109/32.962562>

[9] LiuLiu He, Yang Yang, Zheng. Li, and Ruilian Zhao. 2018. The Reward of Reinforcement learning for Test Optimization for Continuous Integration. In *Journal of Software (in Chinese)*.

[10] Z. Li, R. M. Hierons, and M. Harman. 2007. Search Algorithms for Regression Test Case Prioritization. *IEEE Transactions on Software Engineering* 33 (04 2007), 225–237. <https://doi.org/10.1109/TSE.2007.38>

[11] Jingjing Liang, Sebastian Elbaum, and Gregg Rothermel. 2018. Redefining prioritization: continuous prioritization for continuous integration. In *2018 IEEE/ACM 40th International Conference on Software Engineering (ICSE)*. IEEE, 688–698.

[12] Dusica Marijan, Arnaud Gotlieb, and Sagar Sen. 2013. Test Case Prioritization for Continuous Regression Testing: An Industrial Case Study. In *IEEE International Conference on Software Maintenance*. 540–543.

[13] Dusica Marijan and Marius Liaaen. 2016. Effect of time window on the performance of continuous regression testing. In *2016 IEEE International Conference on Software Maintenance and Evolution (ICSM)*. IEEE, 568–571.

[14] Atif Memon, Zebao Gao, Bao Nguyen, Sanjeev Dhandia, Eric Nickell, Rob Siemborski, and John Micco. 2017. Taming Google-scale Continuous Testing. In *Proceedings of the 39th International Conference on Software Engineering: Software Engineering in Practice Track (ICSE-SEIP '17)*. IEEE Press, Piscataway, NJ, USA, 233–242. <https://doi.org/10.1109/ICSE-SEIP.2017.16>

[15] Daniel Di Nardo, Nadia Alshahwan, Lionel Briand, and Yvan Labiche. 2015. Coverage-based regression test case selection, minimization and prioritization: a case study on an industrial system. *Software Testing Verification & Reliability* 25, 4 (2015), 371–396.

[16] Ali Parsai, Quinten David Soetens, Alessandro Murgia, and Serge Demeyer. 2014. Considering Polymorphism in Change-Based Test Suite Reduction. In *Agile Methods. Large-Scale Development, Refactoring, Testing, and Estimation*, Torgeir Dingsøyr, Nils Brede Moe, Roberto Tonelli, Steve Counsell, Cigdem Gencel, and Kai Petersen (Eds.). Springer International Publishing, Cham, 166–181.

[17] Xiao Qu, Myra B Cohen, and Katherine M Woolf. 2007. Combinatorial interaction regression testing: A study of test case generation and prioritization. In *2007 IEEE International Conference on Software Maintenance*. IEEE, 255–264.

[18] Gregg Rothermel and Mary Jean Harrold. 1997. A Safe, Efficient Regression Test Selection Technique. *ACM TRANSACTIONS ON SOFTWARE ENGINEERING AND METHODOLOGY* 6 (1997), 173–210.

[19] August Shi, Suresh Thummala, Shuvendu K Lahiri, Nikolaj Björner, and Jacek Czerwonka. 2017. Optimizing test placement for module-level regression testing. In *Proceedings of the 39th International Conference on Software Engineering*. IEEE Press, 689–699.

[20] Quinten David Soetens, Serge Demeyer, and Andy Zaidman. 2013. Change-based test selection in the presence of developer tests. In *2013 17th European Conference on Software Maintenance and Reengineering*. IEEE, 101–110.

[21] Helge Spieker, Arnaud Gotlieb, Dusica Marijan, and Morten Mossige. 2017. Reinforcement learning for automatic test case prioritization and selection in continuous integration. In *Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis*. ACM, 12–22.

[22] Shin Yoo, Robert Nilsson, and Mark Harman. 2011. Faster fault finding at Google using multi objective regression test optimisation. In *8th European Software Engineering Conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering (ESEC/FSE'11)*, Szeged, Hungary.

[23] Lian Yu, Lei Xu, and Wei Tek Tsai. 2010. *Time-Constrained Test Selection for Regression Testing*. Springer Berlin Heidelberg.